

Excellent 🚀 Let's move on to **Module 15: JOINS**.

SQL CLASS NOTE – MODULE 15: JOINS

1. Introduction

In SQL, **JOINS** are used to combine rows from **two or more tables** based on related columns.

👉 In our **SchoolDB**, relationships are:

- Students ↔ Enrollments (via StudentID)
- Courses ↔ Enrollments (via CourseID)

This allows us to link students, courses, and enrollments together.

2. Types of SQL Joins

◆ 1. INNER JOIN

- Returns rows where there is a match in both tables.

```
SELECT Students.StudentID, Students.FirstName, Courses.CourseName
FROM Enrollments
INNER JOIN Students ON Enrollments.StudentID = Students.StudentID
INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID;
```

◆ 2. LEFT JOIN (or LEFT OUTER JOIN)

- Returns **all rows from the left table** and matched rows from the right table.
- If no match, shows `NULL` for missing values.

```
SELECT Students.StudentID, Students.FirstName, Courses.CourseName
FROM Students
LEFT JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
LEFT JOIN Courses ON Enrollments.CourseID = Courses.CourseID;
```

◆ 3. RIGHT JOIN (or RIGHT OUTER JOIN)

- Returns **all rows from the right table** and matched rows from the left table.

```
SELECT Courses.CourseID, Courses.CourseName, Students.FirstName
FROM Courses
RIGHT JOIN Enrollments ON Courses.CourseID = Enrollments.CourseID
RIGHT JOIN Students ON Enrollments.StudentID = Students.StudentID;
```

◆ 4. FULL OUTER JOIN

- Returns all rows when there is a match in either left or right table.
- If one side has no match, it still appears with `NULL`.

(Not all databases support FULL JOIN directly, but it can be simulated with UNION of LEFT and RIGHT joins.)

```
SELECT Students.StudentID, Students.FirstName, Courses.CourseName
FROM Students
FULL OUTER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
FULL OUTER JOIN Courses ON Enrollments.CourseID = Courses.CourseID;
```

◆ 5. SELF JOIN

- A table joins itself.
- Example: Students table (find students in the same grade).

```
SELECT A.FirstName AS Student1, B.FirstName AS Student2, A.GradeLevel
FROM Students A
INNER JOIN Students B
  ON A.GradeLevel = B.GradeLevel
 AND A.StudentID <> B.StudentID;
```

3. Why Use Joins?

- To **combine related data** across multiple tables.
- To avoid **data duplication**.
- To answer **complex queries** like:
 - Which student is enrolled in which course?
 - Which teacher teaches which student?
 - Who has no enrollments yet?

4. Real-Life Use Cases

- **Schools:** Show students with courses and teachers.
- **E-commerce:** Orders joined with customers and products.
- **Banks:** Customers joined with transactions and accounts.
- **Hospitals:** Patients joined with doctors and treatments.

5. Classwork (Practice)

👉 Try these queries:

1. Use an **INNER JOIN** to list all students with the courses they are enrolled in.
2. Use a **LEFT JOIN** to show all students and their courses (including those not enrolled).
3. Use a **RIGHT JOIN** to show all courses with enrolled students.
4. Use a **FULL OUTER JOIN** to show all students and all courses (whether matched or not).
5. Use a **SELF JOIN** on the Students table to list pairs of students in the same grade.
6. Show each **teacher** with the courses they teach using a JOIN.
7. Show each **student** with their **teacher(s)** through the Courses table.

6. Challenge Question

👉 Write a query that lists each student, their course(s), and the teacher of that course.

- Show **all students**, even those not enrolled.

(Hint: Use `LEFT JOIN` from Students → Enrollments → Courses).

✅ That's the **expanded Module 15: JOINS**.

Perfect 👉 Let's solve the **Module 15 Classwork + Challenge** with SQL queries.

✅ Solutions to Module 15: JOINS

1. Use an `INNER JOIN` to list all students with the courses they are enrolled in.

```
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName
FROM Enrollments E
INNER JOIN Students S ON E.StudentID = S.StudentID
INNER JOIN Courses C ON E.CourseID = C.CourseID;
```

2. Use a `LEFT JOIN` to show all students and their courses (including those not enrolled).

```
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName
FROM Students S
LEFT JOIN Enrollments E ON S.StudentID = E.StudentID
LEFT JOIN Courses C ON E.CourseID = C.CourseID;
```

3. Use a **RIGHT JOIN** to show all courses with enrolled students.

```
SELECT C.CourseID, C.CourseName, S.FirstName, S.LastName
FROM Courses C
RIGHT JOIN Enrollments E ON C.CourseID = E.CourseID
RIGHT JOIN Students S ON E.StudentID = S.StudentID;
```

4. Use a **FULL OUTER JOIN** to show all students and all courses (whether matched or not).

(If your DBMS doesn't support FULL JOIN, use UNION of LEFT and RIGHT joins.)

```
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName
FROM Students S
FULL OUTER JOIN Enrollments E ON S.StudentID = E.StudentID
FULL OUTER JOIN Courses C ON E.CourseID = C.CourseID;
```

✓ OR (alternative simulation for MySQL):

```
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName
FROM Students S
LEFT JOIN Enrollments E ON S.StudentID = E.StudentID
LEFT JOIN Courses C ON E.CourseID = C.CourseID
UNION
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName
FROM Students S
RIGHT JOIN Enrollments E ON S.StudentID = E.StudentID
RIGHT JOIN Courses C ON E.CourseID = C.CourseID;
```

5. Use a **SELF JOIN** on Students to list pairs of students in the same grade.

```
SELECT A.FirstName AS Student1, B.FirstName AS Student2, A.GradeLevel
FROM Students A
INNER JOIN Students B
    ON A.GradeLevel = B.GradeLevel
    AND A.StudentID < B.StudentID;
```

6. Show each **teacher** with the courses they teach.

```
SELECT Teacher, CourseName
FROM Courses
ORDER BY Teacher;
```

7. Show each **student** with their **teacher(s)** through the Courses table.

```
SELECT S.FirstName, S.LastName, C.CourseName, C.Teacher
FROM Students S
INNER JOIN Enrollments E ON S.StudentID = E.StudentID
INNER JOIN Courses C ON E.CourseID = C.CourseID;
```



Challenge Question

👉 List each student, their course(s), and the teacher of that course.

- Show **all students**, even those not enrolled.

```
SELECT S.StudentID, S.FirstName, S.LastName,  
       C.CourseName, C.Teacher  
FROM Students S  
LEFT JOIN Enrollments E ON S.StudentID = E.StudentID  
LEFT JOIN Courses C ON E.CourseID = C.CourseID  
ORDER BY S.StudentID;
```

✚ This uses **LEFT JOIN** to ensure that even students without enrollments appear, with `NULL` for course and teacher.

✅ That completes **Module 15: JOINS**.